

数据结构实验报告

实验名称：约瑟夫 (Josephus) 问题的五种解决实现方法及其（时间）效率的研究

专业班级	软件工程 1 班
姓名	海子亮
学号	2220222043
完成日期	2023 年 12 月 20 日

一、需求分析

1、实验目的和要求

通过使用五种不同的数据结构和算法解决约瑟夫问题有以下方面的目的和要求：

1. 效率：不同的数据结构和算法对于问题的解决可能具有不同的时间和空间复杂度。选择合适的数据结构和算法可以提高解决问题的效率，特别是在处理大规模问题时。

2. 空间优化： 有些数据结构能够在解决问题时更有效地利用内存空间。选择适当的数据结构可以减小问题的空间复杂度，这对于内存受限的环境或大规模问题尤为重要。

3. 可读性：不同的算法和数据结构可能导致代码的可读性和可维护性不同。选择清晰、易于理解的实现方式将有助于团队协作和代码维护。

4. 通用性： 有些算法和数据结构可能更适合解决一类问题，而不适用于另一类。选择通用性较强的方法可以使解决方案更易于扩展和应用到其他类似的问题。

5. 适应不同约瑟夫问题变体： 约瑟夫问题有多种变体，可能涉及不同的规则、约束或要求。选择不同的数据结构和算法可以使解决方案更容易适应不同的问题变体。

总体而言，选择不同数据结构和算法解决约瑟夫问题的目的是在问题的具体背景和要求下，找到一个既高效、节省空间，又易于理解和具有通用性的解决方案。

（1）输入和输出的形式

输入：

输入形式：以 “in.txt” 的文件形式输入。

其中格式为：每行是用空格分开的两个整数，第一个是 n ，第二个是 m ($m <$

n , $n < 10000$)。最后一行是：0 0 以结束程序。

输出：

输出形式：以“out.txt”的文件形式输出。

其中格式为：

第 i 组结果：

输入数据： n m

数组实现： x. xxxxs

链表实现： x. xxxxs

队列实现： x. xxxxs

反向迭代： x. xxxxs

正向递归： x. xxxxs

其中第 i 组结果与第 i 组（第 i 行）输入数据相对应，结果则为对应的运行时间。

(2) 程序功能

通过使用五种不同的数据结构和算法解决约瑟夫问题，并且计算出相应的函数调用运行时间（时间效率）。

(3) 测试用例设计

考虑到两个输入参数 n , m 的取值要求 ($m < n$, $n < 10000$)，本实验设置了两组相互对照并可自身前后对照的测试用例；一组 n 固定， m 值按一定规律增大；另一组 m 固定， n 值按一定规律增大，如此设置多组测试数据用例。部分如下所示：

```
9 3
99 3
999 3
9 33
9 333
9 3333
0 0
```

二、概要设计

1. 主程序流程：

- 打开输入文件（in.txt）和输出文件（out.txt）。
- 使用循环从输入文件中读取一组输入数据 n 和 m ，直到读取到 n 或 m 为 0 的组合（结束命令）为止。

- 对每一组 n 和 m ，调用不同的解法函数，并记录其运行时间。
- 将每组的结果和运行时间写入输出文件“out.txt”。
- 重复上述步骤直到输入文件结束。
- 关闭文件。

2. 函数调用关系：

- ``main`` 函数是程序的入口，负责文件的读取、写入以及各解法函数的调用。
- ``elapsed_time`` 和 ``elapsed_time1`` 函数用于计算函数运行时间。
- ``array_find``、``l1ist_find``、``queue_find``、``iteration_find``、``recursion_find`` 是五个解法函数，分别采用数组、链表、队列、反向迭代和正向递归的方式解决约瑟夫环问题。
- 在每组 n 和 m 的循环中，分别调用这五个函数，并记录它们的运行时间。
- 运行时间和结果被写入输出文件“out.txt”。

3. 解法函数简要说明：

- ``array_find``：使用数组模拟约瑟夫环，每次找到第 m 个未被删除的元素。
- ``l1ist_find``：使用循环链表模拟约瑟夫环，每次找到第 m 个未被删除的元素。
- ``queue_find``：使用队列模拟约瑟夫环，每次将队首元素移到队尾，直到找到第 m 个未被删除的元素。
- ``iteration_find``：使用反向迭代的方式解决约瑟夫环问题。
- ``recursion_find``：使用正向递归的方式解决约瑟夫环问题。

4. 文件处理：

- ``fi`` 和 ``fo`` 是用于调试时的文件指针，将输入和输出重定向到文件中。
- 在正常运行时，程序通过标准输入和标准输出进行输入和输出。

这个程序的目的是比较不同解法的性能，通过运行时间的比较来评估它们的效率。

三、详细设计

数据类型定义：

1、 主程序：

在主程序中没有定义特定的数据类型。

2、函数：'array_find'

```
int array_find(int n, int m);
```

3、函数：'Llist_find'

```
struct node {  
    int val;  
    struct node* next;  
};
```

```
typedef struct header {  
    int length;  
    struct node* next;  
} head;
```

```
void Llist_find(int n, int m);
```

4、函数：'queue_find'

```
#include <queue>
```

```
queue<int> res;
```

```
void queue_find(int n, int m);
```

5、函数：'iteration_find'

```
int iteration_find(int n, int m);
```

6、函数：'recursion_find'

```
int recursion_find(int n, int m);
```

算法伪代码:

1、主程序:

Read input from "in.txt"

Open output file "out.txt"

Initialize variables n, m, i

Initialize variables et1, et2, et3, et4, et5

While reading n and m from the input file :

If n == 0 or m == 0, break the loop

Calculate et1, et2, et3, et4, et5 using `elapsed_time` function for each method

Write results to the output file

Close files

2、函数：'array_find'

Initialize integer **array** arr of size 1000

Initialize variables count, i, k

While count is **not** n - 1:

Increment i

If i is greater than n, set i to 1

If arr[i] is 0 :

Increment k

If k is equal to m :

Set arr[i] to 1

Increment count

Set k to 0

Find the first index i **where** arr[i] is 0 **and return** i

3、函数：'Llist_find'

Create a circular linked list head

For i from n to 1:

Insert i at the beginning of the list

Initialize variables temp to the first node, count to 1

While the list has more than one node :

If count is equal to m :

Delete the current node

Set temp to the previous node

Set count to 1

Increment count

Move temp to the next node

Print the remaining node **in** the list

4、函数：'queue_find'

Create an empty queue res

For i from 1 to n :

Enqueue i into res

```
Initialize cnt to 0
```

```
While res is not empty :
```

```
    For i from 1 to m - 1 :
```

```
        Enqueue the front element to the rear and dequeue
```

```
        Dequeue the front element
```

5、函数：'iteration_find'

```
Initialize thelast to 0
```

```
For i from 2 to n :
```

```
Update thelast : (thelast + m) % i
```

```
Return thelast
```

6、函数：'recursion_find'

```
Initialize thelast to 0
```

```
If n is 1, return 0
```

```
Return(recursion_find(n - 1, m) + m) % n
```

四、调试分析

1、编辑代码、编译运行所遇到的问题及其解决办法:

(1) 问题：在设计与编辑'解法二：链表实现'时遇到了编译错误和结果错误的问题。

解决方法：通过查询相关学习资料与文献，改正了算法的错误，最终调试通过，结果正确。

(2) 问题：'elapsed_time'函数调用五种算法解决函数时遇到了不兼容的问题。

解决方法：通过修改五种解决函数，使其适配'elapsed_time'函数的调用。

(3) 问题：在某些测试用例下(如：9999 3)运行程序会遇到输出文件(out.txt)中无结果的情况。

解决方法：因为这部分(比较极端的)测试用例在运行程序时会出错(报错)，故修改有问题的测试用例，使其能够正常运行程序。

(4) 问题：在一些测试用例下，'解法四：反向迭代'和'解法五：正向递归'的运行时间非常之小(时常为0.0000s)，而无法进行相关分析。

解决方法：考虑到使用比较极端的测试用例会使其余解法报错，故为解法四与解法五额外单独设置部分测试用例，以获得更加全面的数据，从而进行相关分析。

2、对设计与实现的讨论和分析、算法的时空分析以及心得体会

解法一：数组实现

时间复杂度分析：

1. 算法思路：

- 使用一个大小为 `n` 的数组模拟约瑟夫环，通过循环找到第 `m` 个元素并标记，直到找到第 `n-1` 个标记元素。

2. 时间复杂度：

- 最坏情况下，每次循环中可能需要进行 `m` 次操作。
- 循环次数为 `n`，所以总的时间复杂度为 $O(n * m)$ 。

空间复杂度分析：

1. 数据结构：

- 使用了一个大小为 `n` 的整型数组 `arr`。

2. 空间复杂度：

- 数组的大小为 `n`，因此空间复杂度为 $O(n)$ 。

总结：

- 该算法的时间复杂度为 $O(n * m)$ ，主要由循环的次数决定。
- 空间复杂度为 $O(n)$ ，主要由数组的大小决定。
- 该算法的性能在循环次数较大时可能较低，特别是当 `m` 接近 `n` 时，因为每次都需要做很多次循环。

心得体会：

- 该算法可能在一些情况下效率较低，尤其是当 `m` 的值较大时，因为每次都需要做多次循环，不够高效。
- 如果对性能有要求，可以考虑其他解法，例如使用链表、队列、反向迭代或正向递归等方法进行优化。

解法二：链表实现

时间复杂度分析：

1. 算法思路：

- 使用循环链表模拟约瑟夫环，每次删除第 `m` 个节点，直到链表中只剩一个节点。

2. 时间复杂度：

- 最坏情况下，每次循环中可能需要进行 `m` 次删除操作。
- 循环次数为 `n`，所以总的时间复杂度为 $O(n * m)$ 。

空间复杂度分析：

1. 数据结构：
 - 使用了一个循环链表，其中包含 n 个节点。
2. 空间复杂度：
 - 循环链表的大小为 n ，因此空间复杂度为 $O(n)$ 。

总结：

- 该算法的时间复杂度为 $O(n * m)$ ，主要由循环的次数和每次删除操作的复杂度决定。
- 空间复杂度为 $O(n)$ ，主要由循环链表的大小决定。

心得体会：

- 与数组实现相比，链表实现可能在某些情况下具有更好的性能，尤其是在删除操作的效率较高时。
- 由于链表的插入和删除操作相对较为灵活，可能在某些场景下比数组实现更具优势。

解法三：队列实现

时间复杂度分析：

1. 算法思路：
 - 使用队列模拟约瑟夫环，每次将队首元素放到队尾 $m-1$ 次，然后出队，直到队列为空。
2. 时间复杂度：
 - 最坏情况下，每次循环中可能需要进行 $m-1$ 次入队和出队操作。
 - 循环次数为 n ，所以总的时间复杂度为 $O(n * m)$ 。

空间复杂度分析：

1. 数据结构：
 - 使用了一个队列，队列的大小为 n 。
2. 空间复杂度：
 - 队列的大小为 n ，因此空间复杂度为 $O(n)$ 。

总结：

- 该算法的时间复杂度为 $O(n * m)$ ，主要由循环的次数和每次入队和出队操作的复杂度决定。
- 空间复杂度为 $O(n)$ ，主要由队列的大小决定。

心得体会：

- 与数组和链表实现相比，队列实现可能在某些情况下更为简洁和高效。
- 队列实现适用于需要频繁进行入队和出队操作的场景，但在删除操作的效率上可能不如链表实现。

解法四：反向迭代

时间复杂度分析：

1. 算法思路：
 - 使用反向迭代计算最后一个被剩下的元素的位置。
2. 时间复杂度：
 - 循环 `for` 中的每一次迭代都只需要进行一次计算操作。
 - 循环次数为 `n-1`，所以总的时间复杂度为 $O(n)$ 。

空间复杂度分析：

1. 数据结构：
 - 使用了常量个数的变量，不随输入规模变化而改变。
2. 空间复杂度：
 - 由于只使用了常量个数的变量，空间复杂度为 $O(1)$ 。

总结：

- 该算法的时间复杂度为 $O(n)$ ，主要由循环的次数决定。
- 空间复杂度为 $O(1)$ ，没有额外使用与输入规模相关的数据结构。

心得体会：

- 该算法在时间和空间上相对较优，适用于需要计算最后一个被剩下的元素位置的场景。
- 在约瑟夫环问题中，反向迭代的解法具有较好的性能，特别是相较于某些需要模拟环的实现方式。

解法五：正向递归

时间复杂度分析：

1. 算法思路：
 - 使用正向递归计算最后一个被剩下的元素的位置。
2. 时间复杂度：
 - 递归调用的次数为 `n-1`，每次递归调用需要进行一次加法和一次取模操作。
 - 总的时间复杂度为 $O(n)$ 。

空间复杂度分析：

1. 数据结构：
 - 递归调用栈的深度为 `n-1`。
2. 空间复杂度：
 - 递归调用栈的深度为 `n-1`，因此空间复杂度为 $O(n)$ 。

总结：

- 该算法的时间复杂度为 $O(n)$ ，主要由递归调用的次数决定。
- 空间复杂度为 $O(n)$ ，主要由递归调用栈的深度决定。

心得体会：

- 递归实现在空间上可能会比较消耗资源，特别是在递归调用次数较多的情况下。
- 如果对性能有要求，可以考虑使用迭代或其他解法进行优化。

五、用户使用说明

- 1、正常打开程序（本实验以 Visual Studio 2022 为例）；
- 2、在该工程所在文件目录下创建输入文件“in.txt”和输出文件“out.txt”；

名称	修改日期	类型	大小
x64	2023/11/8 13:53	文件夹	
.gitattributes	2023/12/13 19:35	GITATTRIBUTES ...	1 KB
1.out	2023/11/9 8:42	OUT 文件	0 KB
ds_exp.sln	2023/11/8 13:52	Visual Studio Sol...	2 KB
ds_exp.vcxproj	2023/11/9 14:33	VCXPROJ 文件	7 KB
ds_exp.vcxproj.filters	2023/11/9 14:33	VC++ Project Fil...	2 KB
ds_exp.vcxproj.user	2023/11/8 13:52	Per-User Project...	1 KB
ds48PTA.cpp	2023/12/21 10:50	C++ Source	8 KB
in	2023/12/20 22:50	文本文档	1 KB
in1	2023/12/20 22:48	文本文档	1 KB
josephus_ring.cpp	2023/11/9 9:15	C++ Source	8 KB
out	2023/12/21 12:58	文本文档	1 KB

3、在输入文件“in.txt”中按照“一、需求分析”中的输入输入格式填写测试用例（例如下图）；

名称	修改日期	类型	大小	
x64	2023/11/8 13:53	文件夹		9 3
.gitattributes	2023/12/13 19:35	GITATTRIBUTES ...	1 KB	99 3
1.out	2023/11/9 8:42	OUT 文件	0 KB	999 3
ds_exp.sln	2023/11/8 13:52	Visual Studio Sol...	2 KB	9 33
ds_exp.vcxproj	2023/11/9 14:33	VCXPROJ 文件	7 KB	9 333
ds_exp.vcxproj.filters	2023/11/9 14:33	VC++ Project Fil...	2 KB	9 3333
ds_exp.vcxproj.user	2023/11/8 13:52	Per-User Project...	1 KB	9 3333
ds48PTA.cpp	2023/12/21 10:50	C++ Source	8 KB	0 0
in	2023/12/20 22:50	文本文档	1 KB	

in

文件 编辑 查看

9 3
99 3
999 3
9 33
9 333
9 3333
0 0

4、输出结果在同目录下的输出文件“out.txt”中（例如下图）；

排序

查看

预览

名称	修改日期	类型	大小	
x64	2023/11/8 13:53	文件夹		第1组结果:
.gitattributes	2023/12/13 19:35	GITATTRIBUTES ...	1 KB	输入数据: 9 3
1.out	2023/11/9 8:42	OUT 文件	0 KB	数组实现: 0.0000s
ds_exp.sln	2023/11/8 13:52	Visual Studio Sol...	2 KB	链表实现: 0.0000s
ds_exp.vcxproj	2023/11/9 14:33	VCXPROJ 文件	7 KB	队列实现: 0.0000s
ds_exp.vcxproj.filters	2023/11/9 14:33	VC++ Project Fil...	2 KB	反向迭代: 0.0000s
ds_exp.vcxproj.user	2023/11/8 13:52	Per-User Project...	1 KB	正向递归: 0.0000s
ds48PTA.cpp	2023/12/21 10:50	C++ Source	8 KB	-----
in	2023/12/20 22:50	文本文档	1 KB	第2组结果:
in1	2023/12/20 22:48	文本文档	1 KB	输入数据: 99 3
josephus_ring.cpp	2023/11/9 9:15	C++ Source	8 KB	数组实现: 0.0000s
out	2023/12/21 13:07	文本文档	1 KB	链表实现: 0.0000s
				队列实现: 0.0000s
				反向迭代: 0.0000s
				正向递归: 0.0000s

				第3组结果:

六、测试结果

输入：

9 3
99 33
999 333
9999 3333（未通过）
99 3
999 3
9 33
9 333
9 3333
3 9
33 99
333 999
3333 9999
33 9
333 9
3333 9
3 99
3 999
3 9999
3 3
33 33
333 333
3333 3333
9 9
99 99
999 999
9999 9999（未通过）
0 0

输出：

第 1 组结果：
输入数据：9 3
数组实现：0.0000s
链表实现：0.0000s
队列实现：0.0010s
反向迭代：0.0000s

正向递归：0.0000s

第 2 组结果：
输入数据：99 33
数组实现：0.0000s
链表实现：0.0000s
队列实现：0.0010s
反向迭代：0.0000s
正向递归：0.0000s

第 3 组结果：
输入数据：999 333
数组实现：0.0020s
链表实现：0.0050s
队列实现：0.1160s
反向迭代：0.0000s
正向递归：0.0000s

第 4 组结果：
输入数据：99 3
数组实现：0.0000s
链表实现：0.0000s
队列实现：0.0000s
反向迭代：0.0000s
正向递归：0.0000s

第 5 组结果：
输入数据：999 3
数组实现：0.0000s
链表实现：0.0040s
队列实现：0.0010s
反向迭代：0.0000s
正向递归：0.0000s

第 6 组结果：
输入数据：9 33
数组实现：0.0000s
链表实现：0.0000s
队列实现：0.0000s
反向迭代：0.0000s
正向递归：0.0000s

第 7 组结果：
输入数据：9 333
数组实现：0.0000s
链表实现：0.0000s
队列实现：0.0010s
反向迭代：0.0000s
正向递归：0.0000s

第 8 组结果：
输入数据：9 3333
数组实现：0.0000s
链表实现：0.0000s
队列实现：0.0090s
反向迭代：0.0000s
正向递归：0.0000s

第 9 组结果：
输入数据：3 9
数组实现：0.0000s
链表实现：0.0000s
队列实现：0.0000s
反向迭代：0.0000s
正向递归：0.0000s

第 10 组结果：
输入数据：33 99
数组实现：0.0000s
链表实现：0.0000s
队列实现：0.0010s
反向迭代：0.0000s
正向递归：0.0000s

第 11 组结果：
输入数据：333 999
数组实现：0.0030s
链表实现：0.0000s
队列实现：0.1000s
反向迭代：0.0000s
正向递归：0.0000s

第 12 组结果:	反向迭代: 0.0000s	数组实现: 0.0010s
输入数据: 3333 9999	正向递归: 0.0000s	链表实现: 0.0000s
数组实现: 0.3010s	-----	队列实现: 0.0320s
链表实现: 0.1260s	第 17 组结果:	反向迭代: 0.0000s
队列实现: 10.3560s	输入数据: 3 999	正向递归: 0.0000s
反向迭代: 0.0000s	数组实现: 0.0000s	-----
正向递归: 0.0010s	链表实现: 0.0000s	第 22 组结果:
-----	队列实现: 0.0010s	输入数据: 3333 3333
第 13 组结果:	反向迭代: 0.0000s	数组实现: 0.1050s
输入数据: 33 9	正向递归: 0.0000s	链表实现: 0.0670s
数组实现: 0.0000s	-----	队列实现: 3.4200s
链表实现: 0.0000s	第 18 组结果:	反向迭代: 0.0000s
队列实现: 0.0000s	输入数据: 3 9999	正向递归: 0.0000s
反向迭代: 0.0000s	数组实现: 0.0000s	-----
正向递归: 0.0000s	链表实现: 0.0000s	第 23 组结果:
-----	队列实现: 0.0090s	输入数据: 9 9
第 14 组结果:	反向迭代: 0.0000s	数组实现: 0.0000s
输入数据: 333 9	正向递归: 0.0000s	链表实现: 0.0000s
数组实现: 0.0000s	-----	队列实现: 0.0000s
链表实现: 0.0000s	第 19 组结果:	反向迭代: 0.0000s
队列实现: 0.0010s	输入数据: 3 3	正向递归: 0.0000s
反向迭代: 0.0000s	数组实现: 0.0000s	-----
正向递归: 0.0000s	链表实现: 0.0000s	第 24 组结果:
-----	队列实现: 0.0000s	输入数据: 99 99
第 15 组结果:	反向迭代: 0.0000s	数组实现: 0.0000s
输入数据: 3333 9	正向递归: 0.0000s	链表实现: 0.0000s
数组实现: 0.0010s	-----	队列实现: 0.0040s
链表实现: 0.0410s	第 20 组结果:	反向迭代: 0.0000s
队列实现: 0.0090s	输入数据: 33 33	正向递归: 0.0000s
反向迭代: 0.0000s	数组实现: 0.0000s	-----
正向递归: 0.0000s	链表实现: 0.0000s	第 25 组结果:
-----	队列实现: 0.0000s	输入数据: 999 999
第 16 组结果:	反向迭代: 0.0000s	数组实现: 0.0070s
输入数据: 3 99	正向递归: 0.0000s	链表实现: 0.0060s
数组实现: 0.0000s	-----	队列实现: 0.3070s
链表实现: 0.0000s	第 21 组结果:	反向迭代: 0.0000s
队列实现: 0.0000s	输入数据: 333 333	正向递归: 0.0000s

本实验对上述结果进行了数据可视化处理（相关代码见“七、附录”部分），如下图所示：

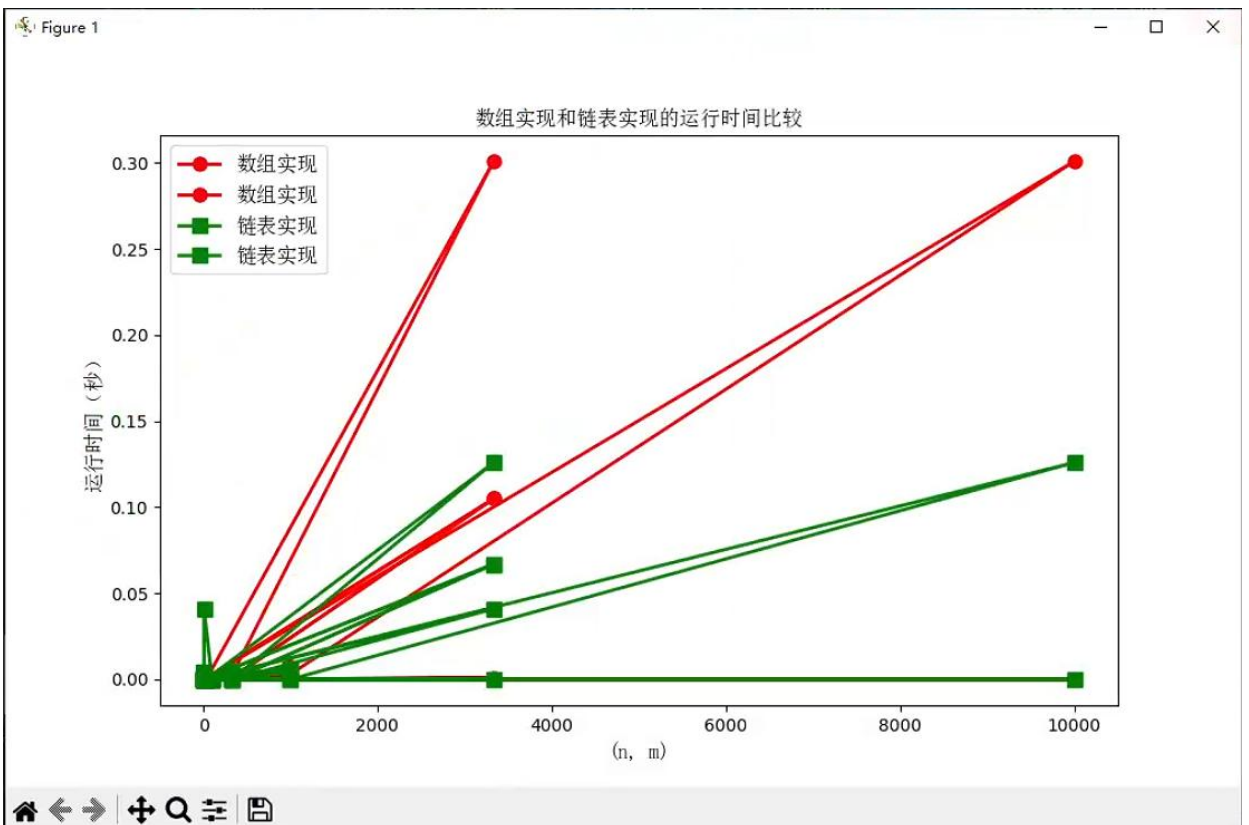
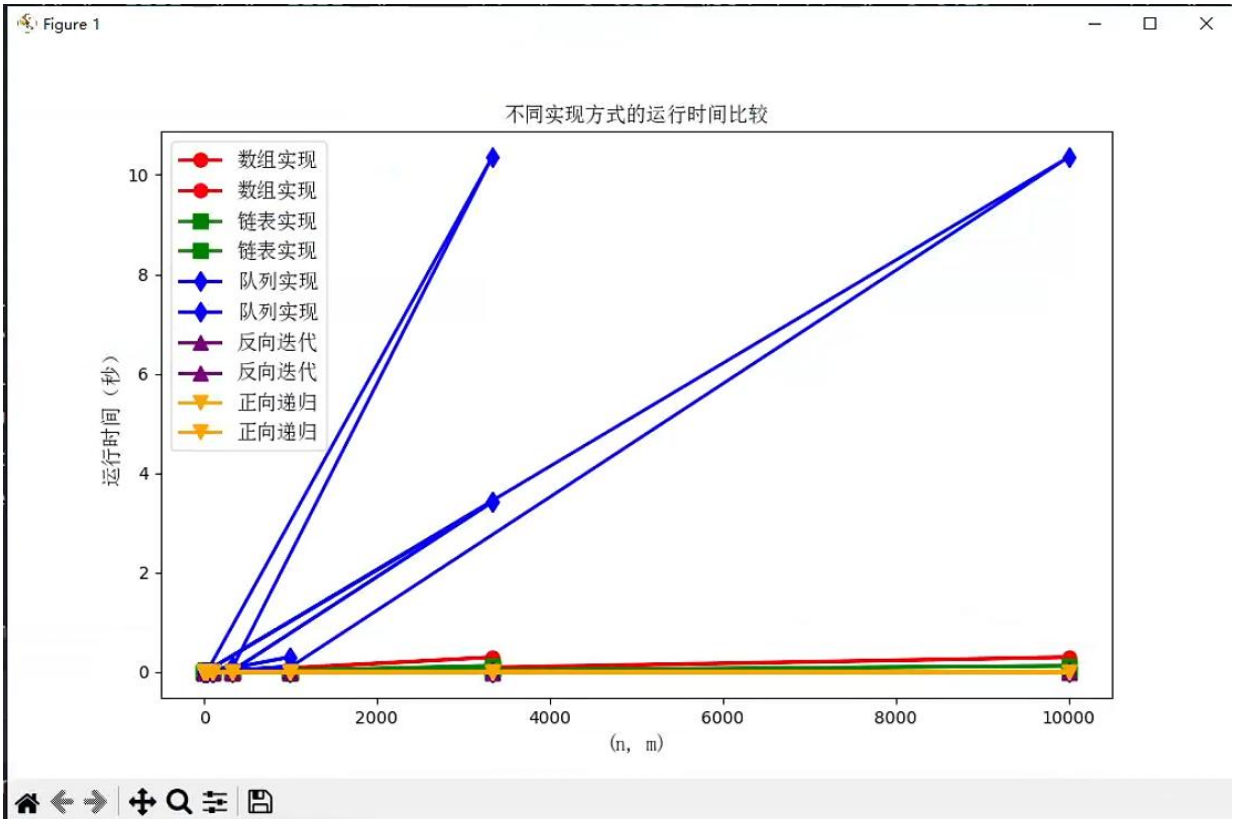


Figure 1

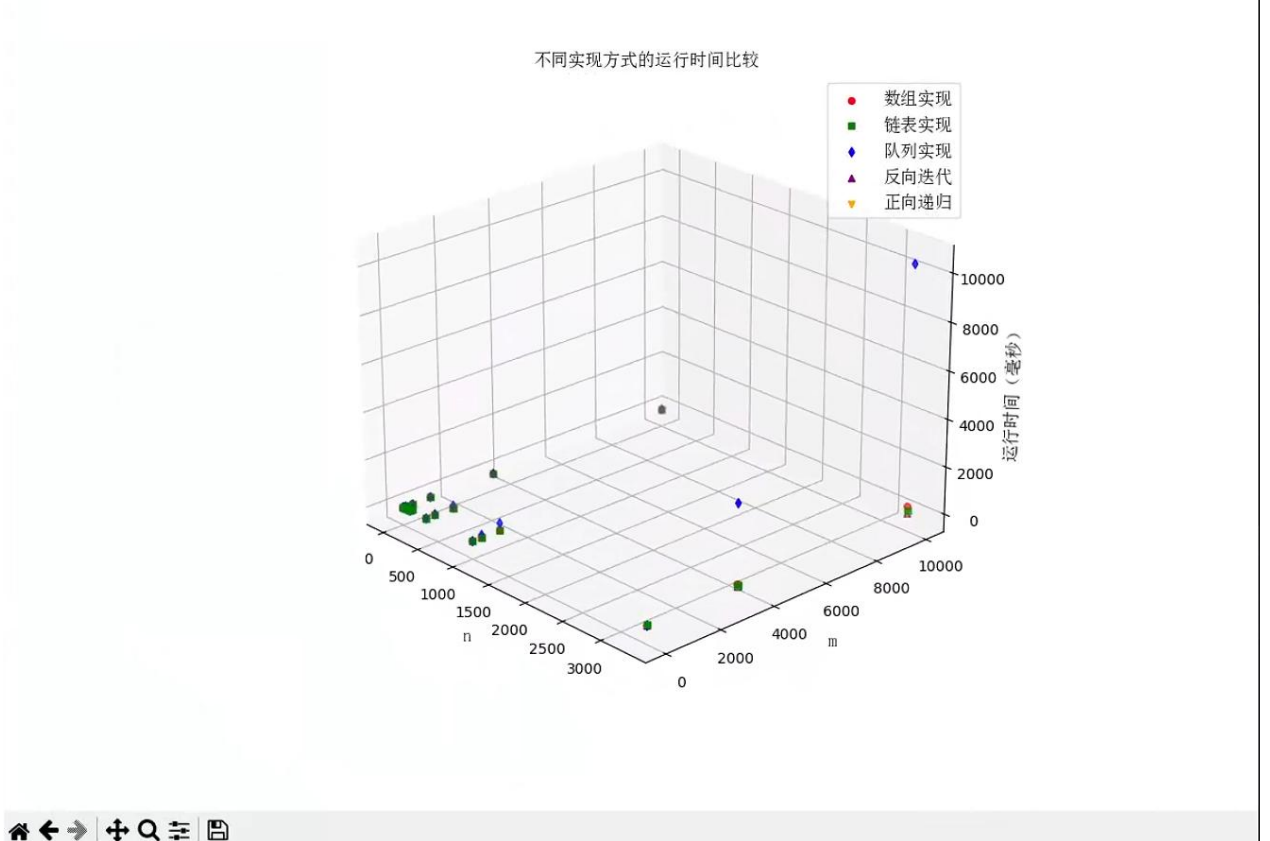
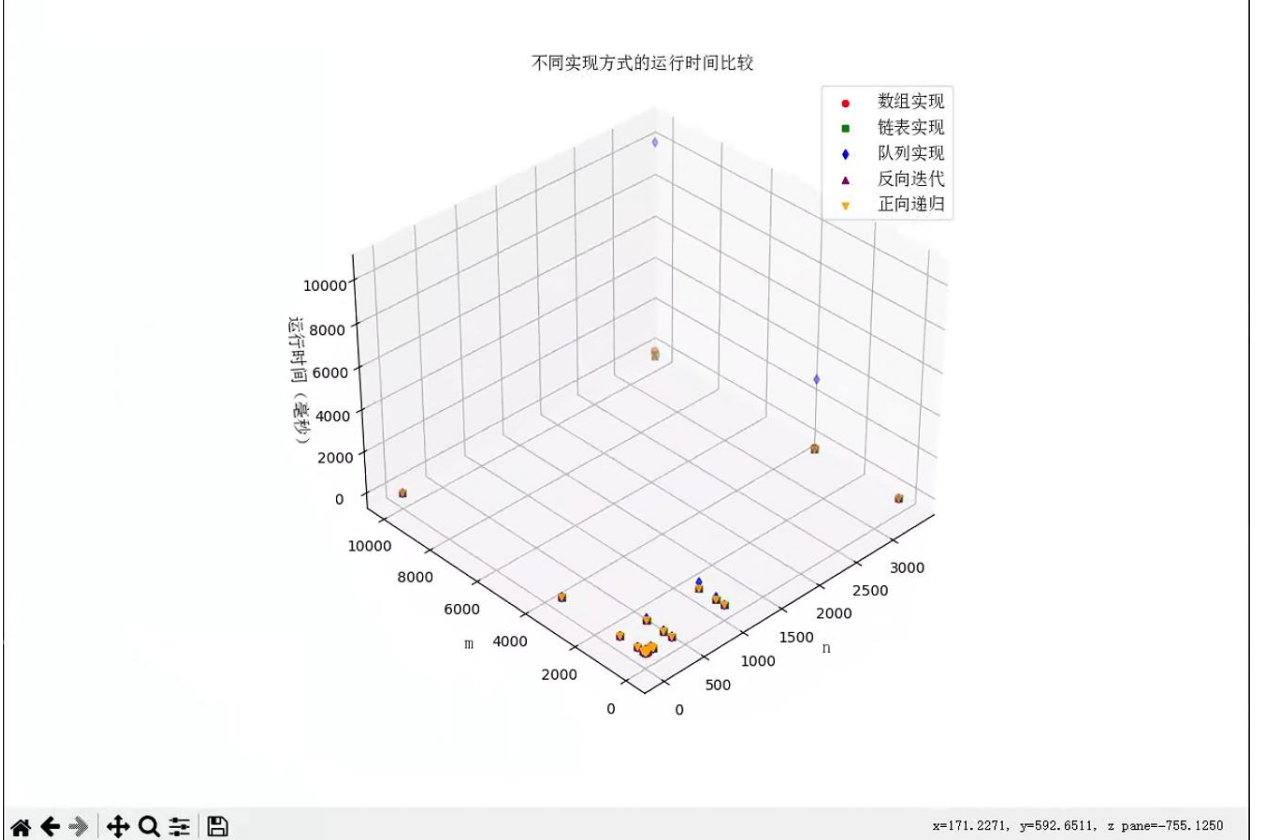


Figure 1



七、附录

源程序清单

```
//[分形的递归输出](https://pintia.cn/problems/1231458003844222976/PROBLEM_DESCRIPTION)
#pragma warning(disable : 4996)
#include <stdio.h>
#include <assert.h>
#include <iostream> // std::cout, std::cin }
#include <fstream> //
fstream::read, ::write, ::getline
#include <string> // std::string
#include <algorithm> // std::transform
#include <cstring> // string.h/memory.h
=> string => cstring::memset, ::memcpy
#include <ctime> // time.h =>
ctime::srand(time(NULL)), ::rand(), ::clock()
#include <vector>
#include <queue>
using namespace std;
//#define DEBUG //提交时注释此行
#ifdef DEBUG
FILE *fi = freopen("1.in", "r", stdin);
FILE *fo = freopen("1.out", "w", stdout);
#define D(x) x
#else
#define D(x)
#endif

//函数调用的运行时间统计(秒)
int array_find(int n, int m);
void Llist_find(int n, int m);
void queue_find(int n, int m);
int iteration_find(int n, int m);
int recursion_find(int n, int m);
//被调用的函数各种各样, 这里举例的接口为
void func(int n, int m);
double elapse_time(int (*f)(int, int), int n, int m)
{
    clock_t start, end;
    start = clock();
    f(n, m);
    end = clock();
    return (double)(end-start)/CLOCKS_PER_SEC;
}

double elapse_time1(void (*f)(int, int), int n, int m)
{
    clock_t start, end;
    start = clock();
    f(n, m);
    end = clock();
    return (double)(end - start) / CLOCKS_PER_SEC;
}

int main()
{
    int n = 0, m = 0, i = 0;
    double et1, et2, et3, et4, et5;
    FILE* f1 = fopen("in.txt", "r");
    FILE* f2 = fopen("out.txt", "w");
    D(sprintf("输入若干组 n 和 m: "));
    while (fscanf(f1, "%d %d\n", &n, &m)) {
        /**/
        if (n == 0 || m == 0) {
            break;
        }
        et1 = elapse_time(array_find, n, m);
        et2 = elapse_time1(Llist_find, n, m);
        et3 = elapse_time1(queue_find, n, m);
        et4 = elapse_time(iteration_find, n, m);
        et5 = elapse_time(recursion_find, n, m);
    }
}
```



```

m);
    fprintf(f2, "第%d 组结果: \n", ++i);
    fprintf(f2, "输入数据: %d %d\n",
n, m);
    fprintf(f2, "数组实现: %-.4fs\n",
et1);
    fprintf(f2, "链表实现: %-.4fs\n",
et2);
    fprintf(f2, "队列实现: %-.4fs\n",
et3);
    fprintf(f2, "反向迭代: %-.4fs\n",
et4);
    fprintf(f2, "正向递归: %-.4fs\n",
et5);
    fprintf(f2,
"-----\n");
    /*/

#ifdef DEBUG
    fclose(fi);
    fclose(fo);
#endif
    return 0;
}
/* 请在这里填写答案 */

//josephus_ring(约瑟夫环问题)

//解法一: 数组实现

int array_find(int n, int m) {

    int arr[10000] = { 0 };
    int count = 0;
    int i = 0, k = 0;

    while (count != n - 1) {
        i++;
        if (i > n) {
            i = 1;
        }
        if (arr[i] == 0) {
            k++;
        }
    }

    if (k == m) {
        arr[i] = 1;
        count++;
        k = 0;
    }
}

for (i = 1; i < n; i++) {
    if (arr[i] == 0) {
        return i;
    }
}
return 0;
}

//解法二: 链表实现

typedef struct header
{
    int length;
    struct node* next;
}header;

typedef struct node
{
    int val;
    struct node* next;
}node;

head* listcreat()
{
    head* p;
    p = (head*)malloc(sizeof(head));
    p->next = NULL;
    p->length = 0;
    return p;
}

void listinsert(head* p, int pos, int x)
{
    if (p == NULL || pos < 0 || pos > p->length)
    {
        printf("listinsert():error\n");
        return;
    }
    node* temp =
(node*)malloc(sizeof(node));

```

```

temp->val = x;
node* pcur = p->next; //指向第一个数据节点
node* plast = p->next; //指向最后一个数据节点
while (pcur != NULL && plast->next != pcur) //使 plast 指向最后一个节点
{
    plast = plast->next;
}
if (p->length == 0) //判断循环链表为空的情况
{
    p->next = temp;
    temp->next = temp;
}
else if (pos == 0) //头插
{
    plast->next = temp;
    temp->next = pcur;
    p->next = temp;
}
else if (pos == p->length) //尾插
{
    plast->next = temp;
    temp->next = pcur;
}
else
{
    node* pval = p->next; //pval 用来指向要插入位置的数据节点
    for (int i = 1; i < pos; i++)
    {
        pval = pval->next;
    }
    temp->next = pval->next;
    pval->next = temp;
}
p->length++;
return;
}

void listdelete(head* p, int x)
{
    node* temp; //temp 指向要删除的节点
    temp = p->next;
    for (int i = 0; i < p->length; i++)
    {
        if (temp->val == x)
        {
            break;
        }
        temp = temp->next;
    }
    node* pcur = p->next; //pcur 指向第一个节点
    node* plast = p->next; //plast 用来指向最后一个节点
    while (plast->next != pcur)
    {
        plast = plast->next;
    }
    if (temp->val != x)
    {
        printf("listprintf():error\n");
        return;
    }
    if (p->length == 1) //只有一个元素时
    {
        p->next = NULL;
    }
    else if (temp == pcur) //删除的是第一个节点
    {
        p->next = pcur->next;
        plast->next = pcur->next;
    }
    else if (temp == plast) //删除的是最后一个节点
    {
        node* pre = p->next; //指向倒数第二个节点
        while (pre->next != plast)
        {
            pre = pre->next;
        }
        pre->next = pcur;
    }
    else

```

```

{
    node* pre = p->next;
    while (pre->next != temp) //使pre指 temp 的前一个节点
    {
        pre = pre->next;
    }
    pre->next = temp->next;
}
p->length--;
}
void listprint(head* p)
{
    if (p == NULL || p->length == 0)
    {
        printf("listprint():error");
        return;
    }
    node* temp = p->next;
    for (int i = 0; i < p->length; i++)
    {
        //printf("%d ", temp->val);
        temp = temp->next;
    }
    //printf("\n");
    return;
}
void Llist_find(int n, int m)
{
    head* p;
    p = listcreat();
    for (int i = n; i > 0; i--)
    {
        listinsert(p, 0, i);
    }
    //listprint(p);
    node* temp = p->next;
    int count = 1;
    //printf("被踢顺序\n");
    while (temp->next != temp) //剩下一个数
    {
        if (count == m)
        {
            node* pre = p->next;
            while (pre->next != temp) //指向
            {
                pre = pre->next;
            }
            //printf("%d ", temp->val);
            listdelete(p, temp->val);
            temp = pre;
            count = 0;
            continue;
        }
        count++;
        temp = temp->next;
    }
    //printf("\n");
    //printf("链表中最后被剩下的是: \n");
    listprint(p);
}
//解法三:队列实现
queue<int> res;

void queue_find(int n, int m) {
    for (int i = 1; i <= n; i++)
    {
        res.push(i);
    }
    int cnt = 0;
    while (!res.empty())
    {
        for (int i = 1; i <= m - 1; i++) //
        {
            res.push(res.front()); //将队首
            res.pop(); //元素放队尾去
        }
        //循环结束后输出队首元素
        //cout << res.front() << " ";
        res.pop(); //出列
    }
}

```

//解法四:反向迭代

```
int iteration_find(int n, int m) {  
    int thelast = 0;  
    for (int i = 2; i <= n; ++i) {  
        thelast = (thelast + m) % i;  
    }  
  
    return thelast;  
}
```

//解法五:正向递归

```
int recursion_find(int n, int m) {  
    int thelast = 0;  
    if (n == 1) {  
        return 0;  
    }  
  
    return ((recursion_find(n - 1, m) + m) %  
n);  
}
```

数据处理可视化 (Python)

```
import matplotlib.pyplot as plt
```

```
from matplotlib.font_manager import FontProperties
```

```
# 设置中文字体
```

```
font = FontProperties(fname=r"C:\Windows\Fonts\simsun.ttc", size=12)
```

```
data = [
```

```
    {"n": 9, "m": 3, "array_time": 0.0000, "l1_time": 0.0000, "queue_time": 0.0010,  
    "iteration_time": 0.0000, "recursion_time": 0.0000},
```

```
    {"n": 99, "m": 33, "array_time": 0.0000, "l1_time": 0.0000, "queue_time": 0.0010,  
    "iteration_time": 0.0000, "recursion_time": 0.0000},
```

```
    {"n": 999, "m": 333, "array_time": 0.0020, "l1_time": 0.0050, "queue_time": 0.1160,  
    "iteration_time": 0.0000, "recursion_time": 0.0000},
```

```
    {"n": 99, "m": 3, "array_time": 0.0000, "l1_time": 0.0000, "queue_time": 0.0000,  
    "iteration_time": 0.0000, "recursion_time": 0.0000},
```

```
    {"n": 999, "m": 3, "array_time": 0.0000, "l1_time": 0.0040, "queue_time": 0.0010,  
    "iteration_time": 0.0000, "recursion_time": 0.0000},
```

```
    {"n": 9, "m": 33, "array_time": 0.0000, "l1_time": 0.0000, "queue_time": 0.0000,  
    "iteration_time": 0.0000, "recursion_time": 0.0000},
```

```
    {"n": 9, "m": 333, "array_time": 0.0000, "l1_time": 0.0000, "queue_time": 0.0010,  
    "iteration_time": 0.0000, "recursion_time": 0.0000},
```

```
    {"n": 9, "m": 3333, "array_time": 0.0000, "l1_time": 0.0000, "queue_time": 0.0090,  
    "iteration_time": 0.0000, "recursion_time": 0.0000},
```

```
    {"n": 3, "m": 9, "array_time": 0.0000, "l1_time": 0.0000, "queue_time": 0.0000,  
    "iteration_time": 0.0000, "recursion_time": 0.0000},
```

```
    {"n": 33, "m": 99, "array_time": 0.0000, "l1_time": 0.0000, "queue_time": 0.0010,  
    "iteration_time": 0.0000, "recursion_time": 0.0000},
```

```
    {"n": 333, "m": 999, "array_time": 0.0030, "l1_time": 0.0000, "queue_time": 0.1000,
```

```

"iteration_time": 0.0000, "recursion_time": 0.0000},
    {"n": 3333, "m": 9999, "array_time": 0.3010, "llist_time": 0.1260, "queue_time": 10.3560,
"iteration_time": 0.0000, "recursion_time": 0.0010},
    {"n": 33, "m": 9, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0000,
"iteration_time": 0.0000, "recursion_time": 0.0000},
    {"n": 333, "m": 9, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0010,
"iteration_time": 0.0000, "recursion_time": 0.0000},
    {"n": 3333, "m": 9, "array_time": 0.0010, "llist_time": 0.0410, "queue_time": 0.0090,
"iteration_time": 0.0000, "recursion_time": 0.0000},
    {"n": 3, "m": 99, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0000,
"iteration_time": 0.0000, "recursion_time": 0.0000},
    {"n": 3, "m": 999, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0010,
"iteration_time": 0.0000, "recursion_time": 0.0000},
    {"n": 3, "m": 9999, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0090,
"iteration_time": 0.0000, "recursion_time": 0.0000},
    {"n": 3, "m": 3, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0000,
"iteration_time": 0.0000, "recursion_time": 0.0000},
    {"n": 33, "m": 33, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0000,
"iteration_time": 0.0000, "recursion_time": 0.0000},
    {"n": 333, "m": 333, "array_time": 0.0010, "llist_time": 0.0000, "queue_time": 0.0320,
"iteration_time": 0.0000, "recursion_time": 0.0000},
    {"n": 3333, "m": 3333, "array_time": 0.1050, "llist_time": 0.0670, "queue_time": 3.4200,
"iteration_time": 0.0000, "recursion_time": 0.0000},
    {"n": 9, "m": 9, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0000,
"iteration_time": 0.0000, "recursion_time": 0.0000},
    {"n": 99, "m": 99, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0040,
"iteration_time": 0.0000, "recursion_time": 0.0000},
    {"n": 999, "m": 999, "array_time": 0.0070, "llist_time": 0.0060, "queue_time": 0.3070,
"iteration_time": 0.0000, "recursion_time": 0.0000}
]

```

分别提取 n 和 m 的值

```

n_values = [(item["n"], item["m"]) for item in data]
array_time = [item["array_time"] for item in data]
llist_time = [item["llist_time"] for item in data]
queue_time = [item["queue_time"] for item in data]
iteration_time = [item["iteration_time"] for item in data]
recursion_time = [item["recursion_time"] for item in data]

```

绘制第一个图：不同实现方式的运行时间比较

```

plt.figure(figsize=(10, 6))
plt.plot(n_values, array_time, '-o', linewidth=2, markersize=8, label='数组实现', color='red')
plt.plot(n_values, llist_time, '-s', linewidth=2, markersize=8, label='链表实现', color='green')
plt.plot(n_values, queue_time, '-d', linewidth=2, markersize=8, label='队列实现', color='blue')

```

```
plt.plot(n_values, iteration_time, '-^', linewidth=2, markersize=8, label='反向迭代',
color='purple')
plt.plot(n_values, recursion_time, '-v', linewidth=2, markersize=8, label='正向递归',
color='orange')
```

```
plt.xlabel('(n, m)', fontproperties=font)
plt.ylabel('运行时间（秒）', fontproperties=font)
plt.title('不同实现方式的运行时间比较', fontproperties=font)
plt.legend(prop=font) # 设置图例字体
plt.show()
```

绘制第二个图：数组实现和链表实现的运行时间比较

```
plt.figure(figsize=(10, 6))
plt.plot(n_values, array_time, '-o', linewidth=2, markersize=8, label='数组实现', color='red')
plt.plot(n_values, llist_time, '-s', linewidth=2, markersize=8, label='链表实现', color='green')
```

```
plt.xlabel('(n, m)', fontproperties=font)
plt.ylabel('运行时间（秒）', fontproperties=font)
plt.title('数组实现和链表实现的运行时间比较', fontproperties=font)
plt.legend(prop=font) # 设置图例字体
plt.show()
```

```
import matplotlib.pyplot as plt
from matplotlib.font_manager import FontProperties
```

设置中文字体

```
font = FontProperties(fname=r"C:\Windows\Fonts\simsun.ttc", size=12)
```

```
data = [
    {"n": 9, "m": 3, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0010,
    "iteration_time": 0.0000, "recursion_time": 0.0000},
    {"n": 99, "m": 33, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0010,
    "iteration_time": 0.0000, "recursion_time": 0.0000},
    {"n": 999, "m": 333, "array_time": 0.0020, "llist_time": 0.0050, "queue_time": 0.1160,
    "iteration_time": 0.0000, "recursion_time": 0.0000},
    {"n": 99, "m": 3, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0000,
    "iteration_time": 0.0000, "recursion_time": 0.0000},
    {"n": 999, "m": 3, "array_time": 0.0000, "llist_time": 0.0040, "queue_time": 0.0010,
    "iteration_time": 0.0000, "recursion_time": 0.0000},
    {"n": 9, "m": 33, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0000,
    "iteration_time": 0.0000, "recursion_time": 0.0000},
    {"n": 9, "m": 333, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0010,
    "iteration_time": 0.0000, "recursion_time": 0.0000},
```

```

{"n": 9, "m": 3333, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0090,
"iteration_time": 0.0000, "recursion_time": 0.0000},
{"n": 3, "m": 9, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0000,
"iteration_time": 0.0000, "recursion_time": 0.0000},
{"n": 33, "m": 99, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0010,
"iteration_time": 0.0000, "recursion_time": 0.0000},
{"n": 333, "m": 999, "array_time": 0.0030, "llist_time": 0.0000, "queue_time": 0.1000,
"iteration_time": 0.0000, "recursion_time": 0.0000},
{"n": 3333, "m": 9999, "array_time": 0.3010, "llist_time": 0.1260, "queue_time": 10.3560,
"iteration_time": 0.0000, "recursion_time": 0.0010},
{"n": 33, "m": 9, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0000,
"iteration_time": 0.0000, "recursion_time": 0.0000},
{"n": 333, "m": 9, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0010,
"iteration_time": 0.0000, "recursion_time": 0.0000},
{"n": 3333, "m": 9, "array_time": 0.0010, "llist_time": 0.0410, "queue_time": 0.0090,
"iteration_time": 0.0000, "recursion_time": 0.0000},
{"n": 3, "m": 99, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0000,
"iteration_time": 0.0000, "recursion_time": 0.0000},
{"n": 3, "m": 999, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0010,
"iteration_time": 0.0000, "recursion_time": 0.0000},
{"n": 3, "m": 9999, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0090,
"iteration_time": 0.0000, "recursion_time": 0.0000},
{"n": 3, "m": 3, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0000,
"iteration_time": 0.0000, "recursion_time": 0.0000},
{"n": 33, "m": 33, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0000,
"iteration_time": 0.0000, "recursion_time": 0.0000},
{"n": 333, "m": 333, "array_time": 0.0010, "llist_time": 0.0000, "queue_time": 0.0320,
"iteration_time": 0.0000, "recursion_time": 0.0000},
{"n": 3333, "m": 3333, "array_time": 0.1050, "llist_time": 0.0670, "queue_time": 3.4200,
"iteration_time": 0.0000, "recursion_time": 0.0000},
{"n": 9, "m": 9, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0000,
"iteration_time": 0.0000, "recursion_time": 0.0000},
{"n": 99, "m": 99, "array_time": 0.0000, "llist_time": 0.0000, "queue_time": 0.0040,
"iteration_time": 0.0000, "recursion_time": 0.0000},
{"n": 999, "m": 999, "array_time": 0.0070, "llist_time": 0.0060, "queue_time": 0.3070,
"iteration_time": 0.0000, "recursion_time": 0.0000}
]

```

分别提取 n 和 m 的值

```
n_values = [item["n"] for item in data]
```

```
m_values = [item["m"] for item in data]
```

```
array_time = [item["array_time"] for item in data]
```

```
llist_time = [item["llist_time"] for item in data]
```

```
queue_time = [item["queue_time"] for item in data]
```

```

iteration_time = [item["iteration_time"] for item in data]
recursion_time = [item["recursion_time"] for item in data]

# 绘制图：不同实现方式的运行时间比较
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')

# 缩小纵坐标的数量级
array_time = [t * 1000 for t in array_time]
llist_time = [t * 1000 for t in llist_time]
queue_time = [t * 1000 for t in queue_time]
iteration_time = [t * 1000 for t in iteration_time]
recursion_time = [t * 1000 for t in recursion_time]

# 绘制散点图
ax.scatter(n_values, m_values, array_time, c='r', marker='o', label='数组实现')
ax.scatter(n_values, m_values, llist_time, c='g', marker='s', label='链表实现')
ax.scatter(n_values, m_values, queue_time, c='b', marker='d', label='队列实现')
ax.scatter(n_values, m_values, iteration_time, c='purple', marker='^', label='反向迭代')
ax.scatter(n_values, m_values, recursion_time, c='orange', marker='v', label='正向递归')

ax.set_xlabel('n', fontproperties=font)
ax.set_ylabel('m', fontproperties=font)
ax.set_zlabel('运行时间（毫秒）', fontproperties=font)
ax.set_title('不同实现方式的运行时间比较', fontproperties=font)
ax.legend(prop=font)

plt.show()

```